

Express.js Middleware

Middleware are functions that run between the request (req) and response (res). They can modify req/res, end the cycle, or pass control to the next middleware using next(). They are used for logging, parsing, authentication, error handling, serving static files, and more.

```
app.use((req, res, next) => {
  console.log("This runs for every request");
  next(); // pass to next middleware or route
});
```

Types of Middleware

1. Application-level middleware 2. Router-level middleware 3. Built-in middleware (express.json, express.urlencoded, express.static) 4. Third-party middleware (morgan, cors) 5. Error-handling middleware (has 4 params: err, req, res, next).

```
// Example: Error-handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send("Something broke!");
});
```

Uses of Middleware

Common uses of middleware in Express.js: 1. Logging 2. Parsing request bodies 3. Authentication/Authorization 4. Error handling 5. Rate limiting/security 6. Serving static files 7. Injecting custom data into req/res objects.

Authentication Middleware Example (JWT)

Authentication middleware verifies a token before allowing access to protected routes.

```
const jwt = require("jsonwebtoken");

function authMiddleware(req, res, next) {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1];

  if (!token) return res.status(401).json({ msg: "No token provided" });

  try {
    const decoded = jwt.verify(token, "MY_SECRET_KEY");
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(403).json({ msg: "Invalid or expired token" });
  }
}

// Protected route example
app.get("/dashboard", authMiddleware, (req, res) => {
  res.json({ msg: `Welcome ${req.user.username}, this is your dashboard.` });
});
```

Response Status Codes

HTTP status codes tell the client what happened with their request. 1xx → Informational 2xx → Success 3xx → Redirection 4xx → Client Error 5xx → Server Error

```
// Examples
res.status(200).json({ msg: "OK" });
res.status(201).json({ msg: "Created" });
res.status(400).json({ error: "Bad Request" });
res.status(401).json({ error: "Unauthorized" });
res.status(404).json({ error: "Not Found" });
res.status(500).json({ error: "Internal Server Error" });
```

Environment Variables in Express.js

Environment variables are key-value pairs stored outside the code, used for secure and flexible configuration. Commonly used for database URLs, API keys, ports, and secret keys. They are accessed via `process.env` in Node.js.

```
// Load environment variables
require("dotenv").config();

const PORT = process.env.PORT || 8000;
const DB_URL = process.env.DB_URL;
const JWT_SECRET = process.env.JWT_SECRET;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```